
LECTURE NOTES

Lecture 5: First-Order Methods

University

Simon Holm
March - 2026



DEPARTMENT OF MATHEMATICS
AND COMPUTER SCIENCE

Contents

1	Intro	3
2	Gradient Descent	3
2.1	Conjugate Direction	4
2.2	Conjugate Gradient Method	5
2.3	Algorithms for CG	5
3	Accelerated descents using momentum (and variations)	8
3.1	Nesterov Momentum	8
3.2	Adagrad	8
3.3	RMSProp (roots mean square)	8
3.4	AdaDelta	9
3.5	Adam (adaptive moment estimation method)	10
3.6	Adamax	10
3.7	Nadam	10
3.8	Hypergradient Descent	10

1 Intro

This covers ways to select a descent direction.

- first-order methods that rely on gradients
- second-order methods that rely on hessian matrix information

Advantages of first order methods:

- cheap iterations: good for small and large scale optimization
- helpful because easy to warm restart

Limitations of first order methods:

- not hard to find challenging instances for them.
- can converge slowly.

2 Gradient Descent

We know that the steepest descent is the opposite to the gradient

$$-\nabla f(x_k)$$

This is normalized steepest descent

$$d_k = -\frac{\nabla f(x_k)}{\|\nabla f(x_k)\|}$$

Example

Suppose we have

$$f(x) = x_1 x_2^2$$

Then the gradient

$$\nabla f = [x_2^2 \ 2x_1 x_2]$$

for

$$x_k = [1 \ 2] \quad d_{k+1} = -\frac{\nabla f(x_k)}{\|\nabla f(x_k)\|} = \frac{[-4 \ -4]}{\sqrt{16+16}} = \left[-\frac{1}{\sqrt{2}} \ -\frac{1}{\sqrt{2}}\right]$$

```
1 i = 0
2 while i < max_iter:
3     g = grad(f, x) # autograd.grad
4     x_new = x - alpha_step * g
5     x = x_new
6     i += 1
```

 Python

Theorem

The next search directions are orthogonal to the current direction. That is.

Proof

2.1 Conjugate Direction

Conjugate Direction

A set of nonzero vectors $\{d_0, d_1, \dots, d_\ell\}$ is said to be conjugate with respect to the symmetric positive definite matrix A if

$$d_i^T A d_j = 0, \quad \text{for all } i \neq j$$

These are defined as

$$x^* - x_0 = \sigma_0 d_0 + \sigma_1 d_1 + \dots + \sigma_{n-1} d_{n-1}$$

Then we can prove that $\sigma_k = \alpha_k$ (proof in lecture). Then

$$x^* = x_0 + \alpha_0 d_0 + \alpha_1 d_1 + \dots + \alpha_{n-1} d_{n-1}$$

Theorem

Given an arbitrary $x_0 \in \mathbb{R}^n$ and a set of conjugate vectors $\{d_0, d_1, \dots, d_{n-1}\}$ the sequence x_k is generated by

$$x_{k+1} = x_k + \alpha_k d_k$$

where α_k is the analytical solution of $\min_{\alpha} f(x_k + \alpha d_k)$ given by

$$\alpha_k = -\frac{r_k^\top d_k}{d_k^\top A d_k}$$

(aka, **conjugate direction algorithm**) converges to the solution x^* of the linear system and minimization problem in at most n steps.

Lets prove it 🤔:

$$\min_{\alpha} f(x_k + \alpha d_k)$$

The derivative is computed as follows

$$\begin{aligned} \frac{\partial}{\partial \alpha} f(x + \alpha d) &= \frac{\partial}{\partial \alpha} (x + \alpha d)^{\top} A (x + \alpha d) - b^{\top} (x + \alpha d) (+c) \\ &= d^{\top} A (x + \alpha d) - d^{\top} b \\ &= d^{\top} A x + 2\alpha d^{\top} (A x - b) + \alpha d^{\top} A d \end{aligned}$$

Now setting $\frac{\partial}{\partial \alpha} f(x + \alpha d) = 0$ results in

$$\alpha = -\frac{d_k^{\top} (A x_k - b)}{d_k^{\top} A d_k} = -\frac{r_k^{\top} d_k}{d_k^{\top} A d_k}$$

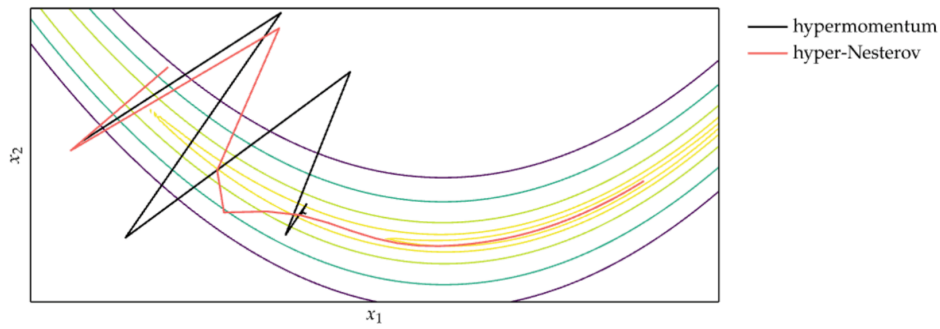


Figure 1: How A changes the geometry of the problem.

2.2 Conjugate Gradient Method

The **conjugate gradient method** is a **conjugate direction method** with the property: In generating its set of conjugate vectors, it can compute a new vector d_k by using only the previous vector d_{k-1} . Hence, little storage and computation requirements.

$$d_k = -r_k + \beta_k d_{k-1}$$

We wish to find

$$\beta_k = \frac{r_k^{\top} A d_{k-1}}{d_{k-1}^{\top} A d_{k-1}}$$

Because of the constraint $d_k^{\top} A d_k = 0$

2.3 Algorithms for CG

Basic CG Algorithm Input: f, x_0 Output: x^*

```

1 | set  $r_0 \leftarrow Ax_0 - b, d_0 \leftarrow r_0, k \leftarrow 0$ ;
2 | while  $r_k \neq 0$  do
3 |    $\alpha_k \leftarrow -\frac{d_k^\top r_k}{d_k^\top A d_k}$ 
4 |    $x_{k+1} \leftarrow x_k + \alpha_k d_k$ 
5 |    $r_{k+1} \leftarrow Ax_{k+1} - b$ 
6 |    $\beta_k \leftarrow \frac{r_{k+1}^\top A d_k}{d_k^\top A d_k}$ 
7 |    $d_{k+1} \leftarrow -r_{k+1} + \beta_{k+1} d_k$ 
8 |    $k \leftarrow k + 1$ 

```

Computationally improved version of the CG Algorithm Input: f, x_0 Output: x^*

```

1 | set  $r_0 \leftarrow Ax_0 - b, d_0 \leftarrow r_0, k \leftarrow 0$ ;
2 | while  $r_k \neq 0$  do
3 |    $\alpha_k \leftarrow -\frac{r(x_k)^\top r_{k(x_k)}}{d_k^\top A d_k}$ 
4 |    $x_{k+1} \leftarrow x_k + \alpha_k d_k$ 
5 |    $r_{k+1} \leftarrow r_k + \alpha_k A d_k$ 
6 |    $\beta_k \leftarrow \frac{r_{k+1}^\top r_{k+1}}{r_k^\top r_k}$ 
7 |    $d_{k+1} \leftarrow -r_{k+1} + \beta_{k+1} d_k$ 
8 |    $k \leftarrow k + 1$ 

```

Major computational cost is the matrix-vector product $A d_k$.

The conjugate gradient method can be applied to nonquadratic functions as well. Since the smooth, continuous function can be locally approximated by a quadratic function, the CG method can be applied to the local approximation. This is called the **nonlinear conjugate gradient method**.

NonLinear Fletcher-Reeves CG Algorithm Input: f, x_0 Output: x^*

```

1 | Evaluate  $f_0 \leftarrow f(x_0), \nabla f_0 = \nabla f(x_0)$ 
2 | Set  $d_0 \leftarrow -\nabla f_0, k \leftarrow 0$ ;
3 | while  $\nabla f_k \neq 0$  do
4 |   Compute  $\alpha_k$  by line search and set  $x_{k+1} \leftarrow x_k + \alpha_k d_k$ ;
5 |   Evaluate  $f_{k+1}$ ;
6 |    $\beta_k^{\text{FR}} \leftarrow \frac{\nabla f_{k+1}^\top \nabla f_{k+1}}{\nabla f_k^\top \nabla f_k}$ 
7 |    $d_{k+1} \leftarrow -\nabla f_{k+1} + \beta_{k+1}^{\text{FR}} d_k$ 
8 |    $k \leftarrow k + 1$ 

```

NonLinear Polak-Ribiere CG Algorithm Input: f, x_0 Output: x^*

```

1 | Evaluate  $f_0 \leftarrow f(x_0), \nabla f_0 = \nabla f(x_0)$ 
2 | Set  $d_0 \leftarrow -\nabla f_0, k \leftarrow 0$ ;
3 | while  $\nabla f_k \neq 0$  do
4 |   Compute  $\alpha_k$  by line search and set  $x_{k+1} \leftarrow x_k + \alpha_k d_k$ ;
5 |   Evaluate  $f_{k+1}$ ;
6 |    $\beta_k^{\text{PR}} \leftarrow \frac{\nabla f_{k+1}^\top (\nabla f_{k+1} - \nabla f_k)}{\nabla f_k^\top \nabla f_k}$ 
7 |    $d_{k+1} \leftarrow -\nabla f_{k+1} + \beta_{k+1}^{\text{PR}} d_k$ 
8 |    $k \leftarrow k + 1$ 

```

PR with $\beta_{k+1}^+ = \max(0, \beta_{k+1}^{\text{PR}})$ becomes PR^+ and guaranteed to converge (satisfies the Wolfe conditions).

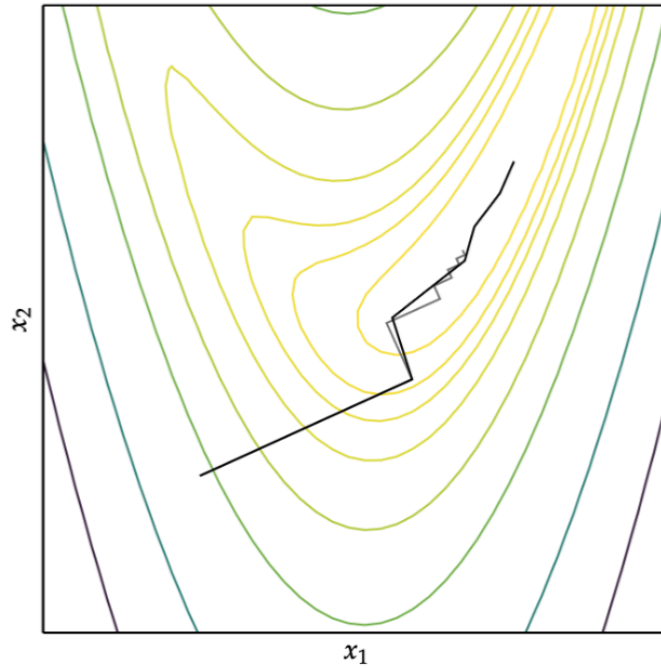


Figure 2: The conjugate gradient method with the Polak-Ribière update. Gradient descent is shown in gray.

3 Accelerated descents using momentum (and variations)

Use momentum of descent to find better minimum

$$x_{k+1} = x_k + v_{k+1}$$

where

$$v_{k+1} = \beta v_k - \alpha \nabla f(x_k).$$

3.1 Nesterov Momentum

Lets slow down enough at the bottom of a valley, do they don't overshoot.

$$x_{k+1} = x_k + v_{k+1}$$

where

$$v_{k+1} = \beta v_k - \alpha \nabla f(x_k + \beta v_k).$$

3.2 Adagrad

$$x_{i,k+1} = s_{i,k} - \frac{\alpha}{\varepsilon + \sqrt{s_{i,k}}} \nabla f_i(x_k)$$

where

$$s_{i,k} = \sum_{j=1}^k (\nabla f_i(x_k))^2$$

and

$$\varepsilon \approx 1 \times 10^{-8}, \quad \alpha = 0.01$$

3.3 RMSProp (roots mean square)

Extends Adagrad to avoid monotonically decreasing learning rate by maintaining a decay-ing average of squared gradients

We can do this by the following

$$x_{i,k+1} = x_{i,k} - \frac{\alpha}{\varepsilon + \text{RMS}(\nabla f_i(x_k))} \nabla f_i(x_k)$$

where

$$\text{RMS}(x) = \sqrt{\frac{1}{n}(x_1^2 + x_2^2 + \dots + x_n^2)}$$

Note that this take the full average of the gradients

We can also use only 2, to prioritize the points where we have reached, usign a parameter γ we can have more control over the momentum rather than just using an average.

$$x_{i,k+1} = x_{i,k} - \frac{\alpha}{\varepsilon + \sqrt{\hat{s}_{i,k}}} \nabla f_i(x_k)$$

where

$$\hat{s}_{k+1} = \gamma \hat{s}_k + (1 - \gamma)(\nabla f(x_k) \odot \nabla f(x_k)), \quad \gamma \in [0, 1]$$

3.4 AdaDelta

extends Adagrad to avoid monotonically decreasing learning rate Modifies RMSProp to eliminate learning rate parameter entirely (e.g the learning rate is set to the non learnable average change of x)

$$x_{i,k+1} = x_{i,k} - \frac{\text{RMS}(\Delta x_i)}{\text{RMS}(\nabla f_i(x_k))} \nabla f_i(x_k)$$

Using AdaDelta, its also common to use

$$x_{i,k+1} = x_{i,k} - \frac{\text{RMS}(\Delta x_i)}{\varepsilon + \sqrt{\hat{s}_{i,k}}} \nabla f_i(x_k)$$

where

$$\hat{s}_{k+1} = \gamma \hat{s}_k + (1 - \gamma)(\nabla f(x_k) \odot \nabla f(x_k)), \quad \gamma \in [0, 1]$$

3.5 Adam (adaptive moment estimation method)

Combination of previous methods, though adam adapts the learning rate to each parameter.

At each iteration the following values are computed

- 1) Biased decaying momentum

$$v_{k+1} = \beta v_k - \alpha \nabla f(x_k)$$

- 2) Biased decaying squared gradient

$$s_{k+1} = \gamma s_k + (1 - \gamma)(\nabla f(x_k) \odot \nabla f(x_k))$$

- 3) Corrected decaying momentum

$$\hat{v}_{k+1} = \frac{v_{k+1}}{1 - \gamma_{v,k}}$$

- 4) Corrected decaying squared gradient

$$\hat{s}_{k+1} = \frac{s_{k+1}}{1 - \gamma_{s,k}}$$

- 5) Finally the next iteration:

$$x_{i,k+1} = x_{i,k} - \frac{\alpha \hat{v}_{k+1}}{\varepsilon + \sqrt{\hat{s}_{k+1}}}$$

This model has default values set to $\alpha = 0.001, \gamma_v = 0.9, \gamma_s = 0.999, \varepsilon = 1 \times 10^{-8}$

3.6 Adamax

Same as Adam, but the bias decaying “squared” gradient is based on the max-norm L_∞ .

$$\begin{aligned} s_{k+1} &= \gamma^\infty s_k + (1 - \gamma^\infty)(\|\nabla f(x_k)\|_\infty) \\ &= \max(\gamma s_k, \|\nabla f(x_k)\|_\infty) \end{aligned}$$

3.7 Nadam

We have seen that Nesterov is often more efficient

Nadam is just Adam which uses the Nesterov momentum

3.8 Hypergradient Descent

Learning rate determines how sensitive the method is to the gradient signal. A good learning rate is important since many accelerated descent methods are highly sensitive to parameters such as learning rate.

We can then use gradient descent to the parameters of an underlying gradient descent.

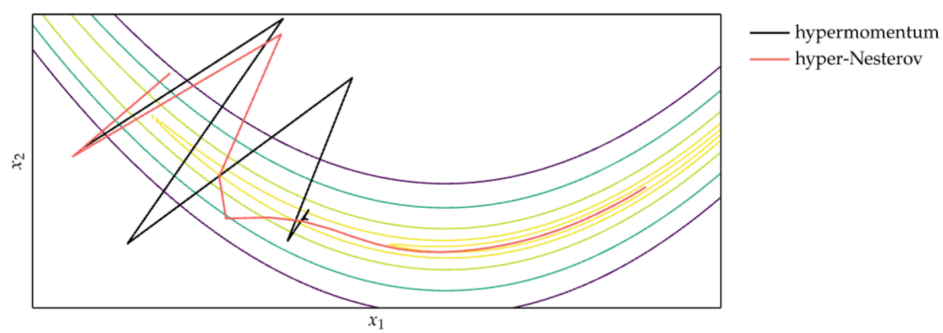


Figure 3: The hypergradient descent method. The learning rate is updated at each iteration using